

Class 2 Notes

System Design: Global Weather Data Platform — Collect • Distribute • Query

1. Problem Statement

Thousands of weather stations are deployed around the world. Each station has sensors that record weather measurements — temperature, humidity, wind speed, pressure, precipitation, and other readings — multiple times a day at regular intervals. Today, every station stores this data on its own local servers; nothing is centralized and nothing is in the cloud.

Many organizations want this data. News agencies want complete, up-to-date weather reports they can feed into their own systems to run weather shows. Application developers and other consumers want a simple API: given a latitude/longitude, return the weather report for that location — for example, the minimum and maximum temperature recorded there.

We want to design a system with three primary responsibilities:

1. **Data collection** — reliably ingest readings from all weather stations into our platform.
2. **Data distribution** — deliver complete weather reports to third-party agencies (news channels, subscribers) across the world, so they can bulk-download the data into their own systems.
3. **Query API** — given a (lat, lng) and query parameters, return the weather report for that location, e.g. min/max temperature, by resolving to the nearest weather station.

Interview tip — restating the problem

In an interview, always restate the problem in your own words like this before designing anything. It confirms your understanding with the interviewer, exposes hidden assumptions (e.g. 'stations keep data on-prem — so collection is part of our job'), and clearly separates the three sub-problems you will design for.

2. Requirements

2.1 Functional Requirements

1. **FR1:** Collect weather data (temperature, humidity, wind speed, etc.) from a large number of weather stations at regular intervals.
2. **FR2:** Distribute full weather reports to multiple agencies across the world, and notify them when new data is available for download.
3. **FR3:** Expose an API that, given a latitude/longitude, returns the weather report for that location — including min and max temperature — based on the nearest weather station.

2.2 Non-Functional Requirements

Attribute	Target	What it means for the design
Scalability	100K weather stations · 10K agencies · 1M API requests/day, peaking at ~1,000 req/sec	Horizontal scaling of ingestion and query tiers; queue to absorb write bursts.
Availability	99.9% ($\approx$ 8.7 hours downtime/year)	Redundancy at every tier, multi-AZ deployment, no single point of failure.
Durability / Retention	Data retained for 2 years	Replicated storage; lifecycle policies to move older data to cheap storage.
Latency	p99 < 2 seconds for the query API	Geospatial index + cache; precomputed aggregates instead of scanning raw readings.
Consistency	Eventual consistency is acceptable	Weather data tolerates minutes of lag; enables async pipelines, caches, and regional replicas.
Workload shape	Read heavy — 100:1 read-to-write ratio	Optimize the read path aggressively: caching, replicas, precomputation.

Interview tip — requirements first

Never draw boxes before agreeing on requirements. The NFRs drive every choice that follows: eventual consistency permits async ingestion; 100:1 reads justify caching and replicas; p99 < 2s rules out scanning raw data at query time.

3. Back-of-the-Envelope Estimation

Assume each station reports every 5 minutes (288 readings/day) and each reading is ~200 bytes after normalization.

Dimension	Calculation	Result
Write throughput	100K stations $\times$ 288 readings/day $\div$ 86,400 sec	$\approx$ 28.8M writes/day $\approx$ 335 writes/sec (avg)
Read throughput	1M requests/day; provision for stated peak	$\approx$ 12 req/sec avg · 1,000 req/sec peak
Raw data growth	28.8M readings/day $\times$ 200 B	$\approx$ 5.8 GB/day $\approx$ 175 GB/month
2-year retention	5.8 GB $\times$ 730 days (before replication/indexes)	$\approx$ 4.2 TB ($\approx$ 12–15 TB with replicas + indexes)
Report distribution	Hourly snapshot ~1–5 GB $\times$ 10K agencies	Large egress — justifies regional edge servers/CDN

Takeaways: the write volume is modest for a partitioned database, the dataset comfortably fits in PostgreSQL with partitioning, but distribution egress is the dominant cost — which is exactly why we replicate reports to regional edge servers rather than serving every download from one origin.

4. API Design

4.1 Query API (for FR3)

```
GET /v1/weather?lat={lat}&lng={lng}&date={YYYY-MM-DD}&metrics=temp_min,temp_max

Response 200:
{
  "station": { "id": "WS-10023", "name": "Bengaluru North", "distance_km": 3.2 },
  "date": "2026-08-02",
  "temp_min_c": 19.4,
  "temp_max_c": 28.7,
  "humidity_avg": 71,
  "wind_speed_max_kmh": 22.5
}
```

Semantics: resolve the nearest active weather station to (lat, lng) using a geospatial index, then return that station's aggregated report for the requested date (default: today). Optional parameters: date range, radius_km cap (return 404 if no station within radius), units.

4.2 Ingestion API (for FR1 — push mode)

```
POST /v1/ingest/readings          (authenticated per station: API key / mTLS)
{
  "station_id": "WS-10023",
  "readings": [
    { "ts": "2026-08-02T10:05:00Z", "temp_c": 26.1, "humidity": 68, "wind_kmh":
14.2 }
  ]
}
Response 202 Accepted  (async processing – eventual consistency)
```

4.3 Distribution APIs (for FR2)

```
POST /v1/subscriptions            – agency subscribes to datasets/regions
GET /v1/reports?date=&region=     – list available report snapshots
Webhook to agency:
{ "event": "report.ready", "report_id": "2026-08-02T10Z-global",
  "download_url": "https://edge.../reports/...?X-Signature=...",  // signed URL
  "expires_at": "2026-08-02T22:00:00Z" }
```

The signed URL is time-limited and scoped to exactly the datasets the agency is entitled to. It points to the regional edge server nearest to the agency, so bulk downloads never hit the origin.

5. High-Level Design

The system decomposes into three pipelines that match the three functional requirements: a write path (collection), a fan-out path (distribution), and a read path (query API). The architecture diagram on the next page shows all three.

5.1 Data Collection (write path)

Stations are heterogeneous — some can call our APIs, some are legacy systems we must poll — so the ingestion gateway supports three source mechanisms:

- **Push (webhook API):** modern stations POST readings to our ingestion endpoint on their own schedule. Simplest for us; requires the station to be able to reach the internet and hold credentials.
- **Pull (collector agents):** our collectors periodically poll legacy stations' servers (HTTP/FTP/custom protocols) on a schedule. We control the cadence; useful when stations cannot push.
- **Batch upload:** stations with poor connectivity upload files periodically (e.g. hourly CSV) to an upload endpoint.

All three paths converge into a message queue (Kafka). The queue decouples ingestion from processing, absorbs bursts (e.g. all stations reporting on the hour), and lets us replay data on failure. Stream processors consume from the queue to validate, deduplicate (idempotency on station_id + timestamp), normalize units, and then write to two destinations:

- **PostgreSQL + PostGIS** — normalized readings and precomputed daily aggregates, serving the query API.
- **Object storage (S3)** — immutable full report snapshots (hourly/daily), which are the unit of distribution to agencies, with a 2-year lifecycle policy.

5.2 Data Distribution (fan-out path)

Agencies need the full weather report — APIs alone don't work for them because they ingest the complete dataset into their own systems to run their shows. The distribution flow works like a deployment rollout:

4. A new report snapshot lands in object storage and emits a 'report ready' event.
5. The distribution/sync service replicates the snapshot to regional edge servers (Americas, Europe, Asia-Pacific) — the same pattern a CDN or a multi-region deployment uses. Sync status per region is tracked in a manifest.
6. Once a region reports 'synced', the notification service notifies every subscribed agency in that region (webhook, with email fallback).
7. The notification carries a signed, time-limited URL scoped to the datasets that agency is entitled to. The agency downloads from its nearest edge server.

Why this design: 10K agencies pulling multi-GB reports would crush a single origin; edge replication moves bytes close to consumers, cuts cross-continent egress, and makes downloads fast and reliable. Signed URLs give per-agency access control without proxying the download through our services.

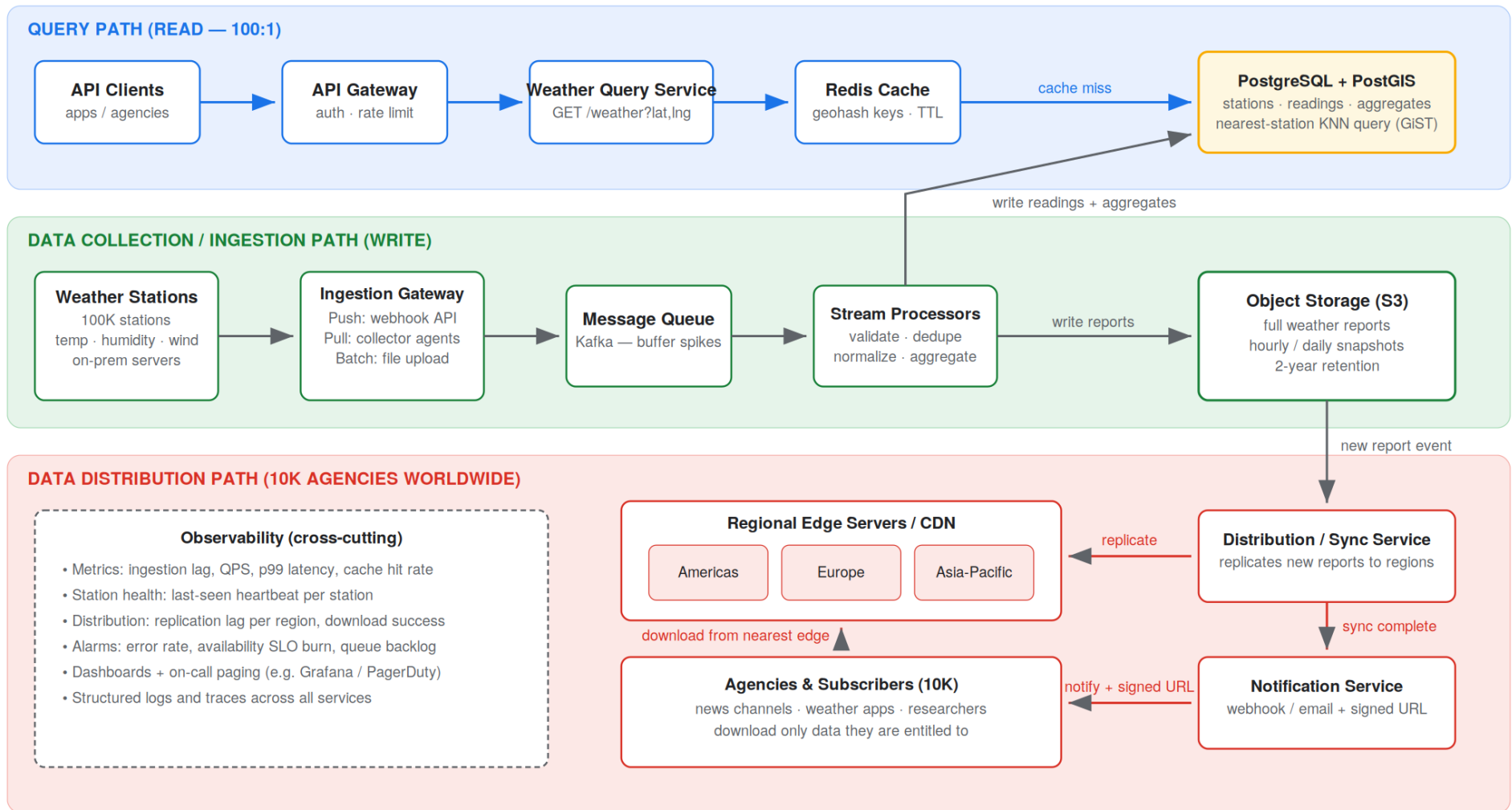
5.3 Query API (read path)

Requests hit an API gateway (authentication, rate limiting) and then the weather query service. The service first checks a Redis cache keyed by a geohash of the (lat, lng) rounded to a cell (e.g. geohash precision 5 ≈ 5 km) plus the date — nearby queries share cache entries, which is ideal for a 100:1 read-heavy workload. On a miss, it runs a nearest-neighbor query in PostgreSQL using PostGIS: find the closest active station to the point, then read that station's precomputed daily aggregate (min/max temperature etc.). Results are cached with a short TTL (e.g. 5–10 minutes) — acceptable because we only promise eventual consistency.

This is how we meet $p99 < 2s$: the hot path is a cache hit; the cold path is one indexed KNN lookup plus one primary-key read of a pre-aggregated row — never a scan over raw readings.

5.4 Architecture Diagram

Weather Data Platform — High-Level Architecture



6. Low-Level Design: Database Schema

Core tables in PostgreSQL (PostGIS extension enabled). Raw readings are partitioned by month so 2-year retention is a cheap partition drop, and the query API reads from the small pre-aggregated table.

```
CREATE EXTENSION postgis;

CREATE TABLE weather_stations (
  station_id    TEXT PRIMARY KEY,
  name          TEXT NOT NULL,
  location      GEOGRAPHY(POINT, 4326) NOT NULL,    -- lat/lng
  altitude_m    REAL,
  status        TEXT NOT NULL DEFAULT 'active',    -- active | inactive
  last_seen_at  TIMESTAMPTZ,                      -- heartbeat for monitoring
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_stations_location ON weather_stations USING GIST (location);

CREATE TABLE readings (                                -- raw time series
  station_id    TEXT NOT NULL REFERENCES weather_stations(station_id),
  ts            TIMESTAMPTZ NOT NULL,
  temp_c        REAL,
  humidity_pct  REAL,
  wind_speed_kmh REAL,
  pressure_hpa  REAL,
  PRIMARY KEY (station_id, ts)                        -- also our dedupe key
) PARTITION BY RANGE (ts);                            -- monthly partitions, drop
after 2 yrs

CREATE TABLE daily_aggregates (                        -- what the query API reads
  station_id    TEXT NOT NULL,
  day           DATE NOT NULL,
  temp_min_c    REAL, temp_max_c REAL, temp_avg_c REAL,
  humidity_avg  REAL, wind_speed_max_kmh REAL,
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (station_id, day)
);

CREATE TABLE agencies (
  agency_id     TEXT PRIMARY KEY,
  name          TEXT NOT NULL,
  region        TEXT NOT NULL,                      -- nearest edge region
  webhook_url   TEXT,
  api_key_hash  TEXT NOT NULL
);

CREATE TABLE subscriptions (
  agency_id     TEXT REFERENCES agencies(agency_id),
  dataset       TEXT NOT NULL,                      -- e.g. 'global-hourly',
  'india-daily' PRIMARY KEY (agency_id, dataset)
);

CREATE TABLE report_snapshots (                      -- distribution manifest
  report_id     TEXT PRIMARY KEY,                   -- e.g. '2026-08-02T10Z-
```

```

global'
  dataset          TEXT NOT NULL,
  object_key       TEXT NOT NULL,                -- S3 key
  regions_synced   TEXT[] NOT NULL DEFAULT '{}',  -- filled by sync service
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

6.1 The Nearest-Station Query (PostGIS)

PostGIS gives us a K-nearest-neighbor operator (<->) that uses the GiST index on location, so finding the closest station is an index lookup, not a table scan:

```

-- Find nearest active station to (lat=12.97, lng=77.59) and its report for a day
SELECT s.station_id, s.name,
       ST_Distance(s.location, ST_MakePoint(77.59, 12.97)::geography)/1000 AS
distance_km,
       a.temp_min_c, a.temp_max_c, a.humidity_avg
FROM   weather_stations s
JOIN   daily_aggregates a ON a.station_id = s.station_id AND a.day = '2026-08-02'
WHERE  s.status = 'active'
ORDER BY s.location <-> ST_MakePoint(77.59, 12.97)::geography -- KNN, uses GiST
index
LIMIT 1;

```

7. Metrics, Monitoring & Alarms

A design is not complete without observability. Define the metrics per pipeline, put them on dashboards, and attach alarms with clear thresholds tied to the NFRs.

Pipeline	Key metrics	Alarms (page on-call when...)
Ingestion	readings ingested/sec; queue (Kafka) lag; validation failure rate; per-station last-seen heartbeat	queue lag > 5 min; validation failures > 1%; > X% of stations silent beyond 3 intervals
Processing / storage	processing latency; DB write errors; replication lag; disk/partition usage	end-to-end ingest latency > 10 min; storage > 80% capacity
Distribution	report build time; per-region sync lag; notification delivery success; download success rate; signed-URL auth failures	region sync lag > 15 min; webhook delivery success < 99%; download errors spike
Query API	QPS; p50/p95/p99 latency; error rate (4xx/5xx); cache hit rate	p99 > 2 s for 5 min; 5xx > 0.1%; cache hit rate < 70%
Availability (SLO)	uptime per service; SLO error-budget burn rate	burn rate implies 99.9% monthly SLO at risk
Cost	egress GB/day; storage growth; compute utilization	egress or storage deviates > 30% from forecast

Complement metrics with structured logs (request IDs propagated across services) and distributed tracing so a slow p99 query can be traced to cache, DB, or network. Dashboards in Grafana (or CloudWatch/Stackdriver), paging via PagerDuty-style on-call rotation.

8. Key Design Decisions & Trade-offs

Decision	Why	Trade-off / alternative
Queue (Kafka) between ingestion and processing	Absorbs top-of-hour bursts; replayable; decouples producers from consumers	Adds latency & operational overhead — fine, we only need eventual consistency
PostgreSQL + PostGIS for queries	Mature geospatial KNN with GiST index; one system for stations + aggregates; 4–15 TB fits partitioned Postgres	At much larger scale, split raw readings into a time-series DB (e.g. Timescale) and keep Postgres for geo lookup
Precomputed daily aggregates	Query path never scans raw readings → meets p99 < 2 s	Aggregates lag raw data by minutes — acceptable under eventual consistency
Edge replication + signed URLs for reports	10K agencies × multi-GB downloads would crush one origin; per-agency scoped access without proxying bytes	Storage duplicated per region — cheaper than cross-continent egress
Geohash-keyed Redis cache	Nearby queries share entries; exploits 100:1 read ratio	Slightly stale results within TTL — acceptable

9. How to Present This in an Interview

A 45-minute system design interview for this problem should flow roughly like this:

8. **Clarify & restate (5 min):** restate the problem, confirm the three responsibilities, ask about scale and consistency expectations.
9. **Requirements (5 min):** write FRs and NFRs on the board exactly as in Section 2 — get the interviewer to agree.
10. **Estimation (3–5 min):** quick back-of-envelope for writes/sec, storage, and egress — shows you design to numbers, not vibes.
11. **API sketch (5 min):** define the query API first (it anchors the read path), then ingestion and distribution contracts.
12. **High-level diagram (15 min):** draw the three pipelines; narrate one request through each. Call out the queue, the two storage systems, edge replication, and signed URLs — and say why.
13. **Deep dive (5–10 min):** go deep where the interviewer steers — schema + PostGIS KNN query, or the distribution manifest, or cache strategy.
14. **Wrap-up (2–3 min):** monitoring/alarms, failure modes, and what you'd do at 10× scale.

Golden rule

Every component in your diagram must be justified by a requirement. If you can't connect a box to an FR or an NFR, remove it. Interviewers reward reasoning, not the number of technologies you name.